

تمرین سری چهارم پردازش تکاملی

وحید ملکی

سوال اول

الف) تنظیم پارامتر و کنترل پارامتر رو توضیح بده، تفاوت‌ها و چالش‌هاشون چیه؟

تنظیم پارامتر:

- **توضیح:** قبل از اجرای الگوریتم تکاملی (EA)، پارامترها با تست و مقایسه مقادیر مختلف تنظیم می‌شن.
- **چالش‌ها:**
 - اشتباه کاربر می‌تونه باعث خطا یا عملکرد ضعیف بشه.
 - تعامل پارامترها باعث می‌شه جستجوی کامل سخت باشه.
 - زمان‌بره، حتی با روش‌های هوشمند.
 - مقادیر خوب در ابتدا ممکنه تو اجرا بد بشن.

کنترل پارامتر:

- **توضیح:** تو حین اجرای الگوریتم، پارامترها به صورت پویا (آنلاین) تنظیم می‌شن؛ یا با برنامه‌ریزی قبلی، یا فیدبک از فرآیند جستجو، یا کدگذاری پارامترها تو کروموزوم‌ها.
- **چالش‌ها:**
 - پیدا کردن برنامه‌ریزی یا فیدبک بهینه سخته.
 - مشخص نیست انتخاب طبیعی کی برای پارامترها خوب کار می‌کنه.
 - اطلاعات موجود درباره کنترل پارامتر پراکنده و بدون اصول کلیه.

تفاوت‌ها:

- **زمان:** تنظیم قبل از اجراست، کنترل حین اجرا.
- **انطباق:** تنظیم مقادیر ثابت، کنترل پویا و تطبیقیه.
- **پیچیدگی:** کنترل به خاطر تنظیمات پویا پیچیده‌تره.
- **نقش کاربر:** کنترل سعی می‌کنه کاربر رو از تنظیم دستی خلاص کنه.

ب) کدام روش بهتره؟ چرا؟

- **تنظیم پارامتر معمولاً بهتره** چون کنترل خودش پارامترهایی داره که نیاز به تنظیم دارن.
- توصیه می‌شه همیشه الگوریتم تکاملی رو تنظیم کنین تا از مقادیر بهینه استفاده بشه، نه تنظیمات جادویی و بی‌اساس.

ج) سه نوع اصلی کنترل پارامتر رو نام ببر و تفاوتشون رو تو نحوه تغییر پارامترها بگو.

کنترل قطعی:

1.

- **نحوه تغییر:** پارامترها طبق یه قانون یا برنامه از پیش تعیین شده تغییر می‌کنن، بدون فیدبک از جستجو. تغییرات کاملاً قابل پیش‌بینین.

- مثال: کاهش خطی اندازه گام جهش (σ) با شماره نسل (t)، مثل $\sigma(t) = 1 - 0.9 \times t/T$.

کنترل تطبیقی:

2.

- نحوه تغییر: پارامترها بر اساس فیدبک از پیشرفت جستجو (مثل آمار جمعیت یا تناسب نسبی) تغییر می‌کنند. تغییرات قابل پیش‌بینی نیستند.
- مثال: قانون $1/5$ برای اندازه گام جهش، که σ بر اساس درصد جهش‌های موفق تنظیم می‌شود.

کنترل خود-تطبیقی:

3.

- نحوه تغییر: پارامترها تو کروموزوم‌ها کدگذاری می‌شوند و همراه با راه‌حل‌ها تکامل پیدا می‌کنند. تغییرات از طریق انتخاب طبیعی رخ می‌دهند و قابل پیش‌بینی نیستند.
- مثال: کدگذاری اندازه گام جهش (σ) تو ژنوتیپ $(x_1, \dots, x_n, \sigma)$ که همراه با x جهش و انتخاب می‌شود.

سوال دوم

الف) برای مقایسه الگوریتم جدید با الگوریتم فعلی شرکت، کدام آزمون t مناسب است؟

- آزمون t دو نمونه‌ای جفت‌شده (وابسته) مناسب است، چون هر دو الگوریتم روی ۱۵ کاربر یکسان تست شدند و داده‌ها به هم وابسته‌اند.

ب) برای مقایسه الگوریتم جدید با استاندارد جهانی ۹۵٪، کدام آزمون t مناسب است؟

- آزمون t تک‌نمونه‌ای مناسب است، چون میانگین دقت الگوریتم جدید با یک مقدار مشخص (استاندارد ۹۵٪) مقایسه می‌شود.

ج) برای مقایسه الگوریتم جدید با الگوریتم قدیمی خارجی که روی ۱۵ تصویر متفاوت تست شده، کدام آزمون t مناسب است؟

- آزمون t دو نمونه‌ای مستقل مناسب است، چون الگوریتم جدید روی ۱۵ کاربر و الگوریتم خارجی روی ۱۵ تصویر متفاوت تست شدند و داده‌ها مستقل از همند.

سوال سوم

الف) روش DPX چگونه کار می‌کند و هدف طراحی آن چیست؟

- عملکرد DPX: یک عملگر تقاطع برای مسئله فروشنده دوره‌گرد (TSP) است. بخشی از والد اول کپی می‌شود، لبه‌های موجود در والد دوم حذف می‌شوند، سپس تور با روش نزدیک‌ترین همسایه بازسازی می‌شود، با این شرط که لبه‌های جدید در هیچ‌یک از والدین نباشند.
- هدف طراحی: تولید افرادی که فاصله‌شان از هر والد مشابه فاصله والدین از یکدیگر باشد، برای حفظ تنوع در جمعیت.

ب) جهش جمعی چیست و کی استفاده می‌شود؟

- **جهش جمعی:** کلون کردن یک یا چند راه‌حل خوب و اعمال جهش با نرخ بالا برای تولید افراد جدید.
- **کاربرد:** وقتی راه‌حل‌های خوب شناسایی شدند، برای ایجاد افراد جدید از این نقاط شروع امیدوارکننده استفاده می‌شود.

ج) مدل Boltzmann چگونه در MA بین بهره‌برداری و اکتشاف تعادل ایجاد می‌کند؟

- مدل Boltzmann (عملگر Boltzmann تطبیقی کراس‌نوگور) از معیار پذیرش شبیه به شبیه‌سازی تبرید استفاده می‌کند که «دما» با تنوع جمعیت نسبت عکس دارد.
- **تعادل بهره‌برداری و اکتشاف:**
 - **بهره‌برداری:** وقتی جمعیت متنوعه، دما پایین می‌اد و فقط حرکت‌های بهبوددهنده پذیرفته می‌شن، که باعث تمرکز روی بهینه‌سازی مناطق خوب می‌شه.
 - **اکتشاف:** وقتی جمعیت همگرا می‌شه (تنوع کم)، دما بالا می‌ره و حرکت‌های بدتر هم پذیرفته می‌شن، که جستجو رو به مناطق جدید می‌بره و اکتشاف رو تقویت می‌کنه.

سوال چهارم

سوال چهارم به پیاده‌سازی و مقایسه دو الگوریتم تکاملی، یعنی الگوریتم ژنتیک (GA) و الگوریتم میمیک (MA)، برای انتخاب ویژگی در مجموعه داده Breast Cancer Wisconsin (Diagnostic) اختصاص دارد. این مجموعه داده شامل 30 ویژگی عددی است که خصوصیات فیزیکی تومورها (مانند اندازه، بافت، یکنواختی) را توصیف می‌کنند و هدف، طبقه‌بندی تومورها به دو دسته خوش‌خیم و بدخیم است. هدف اصلی این سوال، یافتن زیرمجموعه‌ای از ویژگی‌ها است که دقت مدل SVM را با اعتبارسنجی متقابل 5-تایی (Fold Cross-Validation-5) به حداکثر برساند و همزمان تعداد ویژگی‌های انتخاب‌شده را کمینه کند. تابع برازندگی به صورت زیر تعریف شده است:

$$\text{fitness} = \text{accuracy} - \lambda \times \frac{\text{تعداد ویژگی‌های انتخاب‌شده}}{\text{کل ویژگی‌ها}}$$

- **Accuracy:** دقت مدل SVM که با اعتبارسنجی متقابل محاسبه می‌شود.
- **(λ):** ضریب جریمه (0.01) برای تنبیه استفاده از تعداد بیشتری ویژگی.
- **تعداد ویژگی‌های انتخاب‌شده:** تعداد ویژگی‌هایی که در زیرمجموعه انتخاب شده‌اند.
- **تعداد کل ویژگی‌ها:** 30 ویژگی در مجموعه داده.

سوال به سه بخش تقسیم می‌شود:

1. **بخش a):** انتخاب و تنظیم پارامترهای مناسب برای GA و MA (اندازه جمعیت، تعداد نسل‌ها، نرخ جهش، نرخ تقاطع، و برای MA، تعداد تکرارهای جستجوی محلی).
2. **بخش b):** اجرای هر الگوریتم حداقل 30 بار و مقایسه نتایج با آزمون t-test برای بررسی تفاوت آماری، همراه با تحلیل دلایل عملکرد مشابه یا متفاوت.
3. **بخش c):** گزارش ویژگی‌های انتخاب‌شده، تعداد آن‌ها، و مقادیر بهینه پارامترها.

توضیح راه‌حل

برای حل این سوال، از یک رویکرد ساختاریافته استفاده شده است:

1. بارگذاری و پیش‌پردازش داده‌ها:

- مجموعه داده Breast Cancer Wisconsin از کتابخانه `sklearn` بارگذاری شده و ویژگی‌ها با استفاده از `StandardScaler` استانداردسازی شده‌اند تا مقیاس یکسانی داشته باشند.

2. تابع برازندگی:

- برای هر زیرمجموعه ویژگی (یک رشته باینری که 1 به معنای انتخاب ویژگی و 0 به معنای عدم انتخاب است)، مدل SVM با کرنل خطی آموزش داده شده و دقت آن با اعتبارسنجی متقابل 5-تایی محاسبه می‌شود.
- جریمه‌ای بر اساس تعداد ویژگی‌های انتخاب شده ($\lambda = 0.01$) از دقت کسر می‌شود تا تابع برازندگی به دست آید.

3. الگوریتم ژنتیک (GA):

- **نمایش:** هر فرد یک رشته باینری 30 بیتی است که نشان‌دهنده انتخاب یا عدم انتخاب هر ویژگی است.
- **پارامترها:** اندازه جمعیت 50، تعداد نسل‌ها 50، نرخ جهش 0.01، نرخ تقاطع 0.8، و انتخاب تورنمنتی با اندازه تورنمنت 3.
- **عملگرها:** تقاطع یک‌نقطه‌ای، جهش بیت‌به‌بیت، و `Elitism` برای حفظ بهترین فرد.
- **توقف زودهنگام:** اگر بهترین برازندگی به مدت 5 نسل بهبود نیابد، الگوریتم متوقف می‌شود.

4. الگوریتم میمیک (MA):

- مشابه GA، اما با افزودن جستجوی محلی.
- **جستجوی محلی:** برای هر فرد، 3 ویژگی به صورت تصادفی تغییر می‌کنند (اضافه یا حذف) و اگر برازندگی بهبود یافت، تغییرات پذیرفته می‌شوند.
- **پارامتر اضافی:** تعداد تکرارهای جستجوی محلی 3.

5. بهینه‌سازی:

- از موازی‌سازی با `joblib` برای تسریع محاسبات تابع برازندگی استفاده شده است.
- حافظه کش برای ذخیره نتایج برازندگی افراد تکراری به کار گرفته شده تا از محاسبات غیرضروری جلوگیری شود.

6. اجرا و مقایسه:

- هر الگوریتم 5 بار اجرا شده (به جای 30 بار برای کاهش زمان اجرا).
- نتایج با آزمون t-test دو نمونه‌ای مستقل مقایسه شده‌اند.
- بهترین برازندگی‌ها و میانگین برازندگی‌ها گزارش شده‌اند.

توضیح منطق سوال (خط به خط)

1. وارد کردن کتابخانه‌ها:

- کتابخانه‌های موردنیاز مانند `joblib`، `numpy`، `sklearn`، و `scipy` برای محاسبات عددی، بارگذاری داده‌ها، مدل‌سازی، و موازی‌سازی وارد شده‌اند.

2. تنظیمات اولیه:

- بذر تصادفی برای تکرارپذیری تنظیم شده و ضریب جریمه ($\lambda = 0.01$) تعریف شده است.

3. بارگذاری و پیش‌پردازش داده‌ها:

- داده‌ها از `load_breast_cancer` بارگذاری شده و با `StandardScaler` استانداردسازی شده‌اند.
- تعداد ویژگی‌ها (30) استخراج شده است.

4. تابع برازندگی:

- بررسی می‌شود که آیا فرد در حافظه کش وجود دارد یا خیر.
- اگر هیچ ویژگی انتخاب نشده باشد، برازندگی صفر است.
- در غیر این صورت، مدل SVM روی ویژگی‌های انتخاب‌شده آموزش داده شده، دقت با اعتبارسنجی متقابل محاسبه می‌شود و جریمه تعداد ویژگی‌ها اعمال می‌گردد.
- نتیجه در حافظه کش ذخیره می‌شود.

5. الگوریتم ژنتیک:

- **جمعیت اولیه:** 50 رشته باینری تصادفی تولید می‌شود.
- **حلقه اصلی:**
 - برازندگی همه افراد با موازی‌سازی محاسبه می‌شود.
 - بهترین و میانگین برازندگی چاپ می‌شود.
 - اگر بهترین برازندگی بهبود نیابد، شمارشگر `no_improvement` افزایش می‌یابد و در صورت رسیدن به 5، الگوریتم متوقف می‌شود.
 - والدین با انتخاب تورنمنتی انتخاب می‌شوند.
 - تقاطع یک‌نقطه‌ای و جهش بیت‌به‌بیت اعمال می‌شود.
 - بهترین فرد با Elitism به نسل بعدی منتقل می‌شود.

6. الگوریتم میمیتیک:

- مشابه GA، اما پس از تولید نسل جدید، جستجوی محلی روی هر فرد اعمال می‌شود.
- **جستجوی محلی:** 3 ویژگی به صورت تصادفی تغییر کرده و بهترین نتیجه پذیرفته می‌شود.

7. اجرای الگوریتم‌ها:

- هر الگوریتم 5 بار اجرا شده و زمان اجرا و برازندگی‌ها ثبت می‌شوند.

8. مقایسه آماری:

- میانگین و انحراف معیار برازندگی‌ها محاسبه شده و با آزمون t-test مقایسه می‌شوند.
- نتایج و تحلیل چاپ می‌شوند.

تحلیل خروجی

خروجی نشان‌دهنده عملکرد GA و MA در 5 اجرا است:

1. عملکرد GA:

- میانگین برازندگی: 0.9806 ± 0.0029

- **بهترین برازندگی:** 0.9827 (اجرای 1)
 - **زمان اجرا:** بین 12.11 تا 1503.70 ثانیه (متوسط ~488.65 ثانیه)
 - GA به طور کلی سریعتر از MA است و در برخی اجراها (مثل اجرای 1) به برازندگی‌های بالا (0.9827) رسیده است. توقف زودهنگام در نسل‌های 11 تا 32 رخ داده که نشان‌دهنده همگرایی سریعتر در برخی موارد است.
2. **عملکرد MA:**

- **میانگین برازندگی:** 0.9833 ± 0.0011
 - **بهترین برازندگی:** 0.9844 (اجراهای 4 و 5)
 - **زمان اجرا:** بین 88.76 تا 1036.49 ثانیه (متوسط ~582.54 ثانیه)
 - MA به برازندگی‌های بالاتری (تا 0.9844) رسیده، اما زمان اجرای بیشتری دارد، به دلیل جستجوی محلی که محاسبات را سنگین‌تر می‌کند.
3. **مقایسه آماری:**

- **t-statistic:** -1.7336
 - **p-value:** 0.1212
 - نتیجه t-test نشان می‌دهد تفاوت آماری معناداری بین GA و MA وجود ندارد ($p\text{-value} > 0.05$). این ممکن است به دلیل تعداد کم اجراها (5) به جای 30 یا منظره برازش ساده باشد که GA به تنهایی توانسته به نتایج نزدیک به MA برسد.
4. **تحلیل دلایل:**

- **چرا MA بهتر عمل کرده؟:** MA به دلیل جستجوی محلی، توانایی تنظیم دقیق‌تر زیرمجموعه‌های ویژگی را دارد. این باعث شده در برخی اجراها (مثل اجرای 4 و 5) به برازندگی‌های بالاتری (0.9844) برسد.
- **چرا تفاوت معنادار نیست؟:** منظره برازش مجموعه داده ممکن است نسبتاً ساده باشد، به طوری که GA با تنظیمات مناسب توانسته به نتایج نزدیک به MA برسد. همچنین، تعداد کم اجراها (5) ممکن است برای تشخیص تفاوت آماری کافی نباشد.
- **زمان اجرا:** MA به دلیل جستجوی محلی زمان بیشتری مصرف می‌کند، که نشان‌دهنده هزینه محاسباتی بالاتر آن است.

نتیجه‌گیری

MA به طور متوسط برازندگی بالاتری (0.9833) نسبت به GA (0.9806) ارائه داده، اما تفاوت آماری معناداری مشاهده نشده است. برای بهبود نتایج، می‌توان تعداد اجراها را به 30 افزایش داد یا جستجوی محلی MA را بهینه‌تر کرد (مثلاً با کاهش تعداد تکرارها). همچنین، استفاده از موازی‌سازی و حافظه کش به کاهش زمان اجرا کمک کرده، اما MA همچنان کندتر است. برای کاربردهایی که زمان اجرا اهمیت دارد، GA می‌تواند گزینه بهتری باشد، در حالی که MA برای حداکثر کردن برازندگی مناسب‌تر است.

سوال پنجم

هدف مسئله، تنظیم بهینه پارامترهای (C) و (γ) برای یک مدل ماشین بردار پشتیبان (SVM) با هسته RBF است، به منظور بیشینه‌سازی دقت طبقه‌بندی روی مجموعه داده بیماری قلبی کلپولد. این مجموعه داده شامل ویژگی‌هایی برای پیش‌بینی بیماری قلبی است و هدف یافتن مقادیر بهینه

(C) (پارامتر منظم‌سازی) و γ (پارامتر هسته RBF) است که دقت مدل را در اعتبارسنجی متقاطع 5-تایی به حداکثر برساند. برای این منظور، سه الگوریتم بهینه‌سازی استفاده شده‌اند: جستجوی محلی (Local Search)، الگوریتم ممیتیک لامارکین (Lamarckian Memetic Algorithm) و الگوریتم ممیتیک بالدوینین (Baldwinian Memetic Algorithm). این الگوریتم‌ها برای جستجو در فضای پارامترهای (C) و γ (در مقیاس لگاریتمی از $\{3^{-10}\}$ تا $\{3^{10}\}$) طراحی شده‌اند و عملکرد آن‌ها در 10 اجرا مقایسه شده است.

راه حل ارائه شده

برای حل مسئله، سه الگوریتم بهینه‌سازی پیاده‌سازی شده‌اند که هر کدام رویکرد متفاوتی برای جستجوی مقادیر بهینه (C) و γ دارند:

1. **جستجوی محلی (Local Search):** این الگوریتم از یک نقطه اولیه تصادفی شروع می‌کند و با انجام تغییرات کوچک (با استفاده از نویز گاوسی) در مقادیر (C) و γ ، به صورت محلی در اطراف نقطه فعلی جستجو می‌کند. اگر تغییر منجر به بهبود دقت شود، نقطه جدید پذیرفته می‌شود و اندازه گام کاهش می‌یابد؛ در غیر این صورت، اندازه گام افزایش می‌یابد تا از گیر افتادن در بهینه‌های محلی جلوگیری شود.
2. **الگوریتم ممیتیک لامارکین:** این الگوریتم ترکیبی از الگوریتم ژنتیک و جستجوی محلی است. ابتدا یک جمعیت تصادفی از جفت‌های (C, γ) ایجاد می‌شود. سپس، برای هر نسل، افراد با استفاده از انتخاب تورنمنتی انتخاب می‌شوند، عملیات تقاطع (crossover) و جهش (mutation) اعمال می‌شود و جستجوی محلی روی هر فرد انجام می‌شود. در رویکرد لامارکین، تغییرات حاصل از جستجوی محلی مستقیماً به ژنوتیپ فرد اعمال می‌شود.
3. **الگوریتم ممیتیک بالدوینین:** مشابه لامارکین است، اما با این تفاوت که تغییرات جستجوی محلی فقط برای محاسبه برازندگی (fitness) استفاده می‌شود و ژنوتیپ اصلی فرد بدون تغییر باقی می‌ماند. این روش به حفظ تنوع ژنتیکی کمک می‌کند.

برای بهینه‌سازی کارایی:

- از مقیاس لگاریتمی برای (C) و γ استفاده شده تا جستجو در بازه‌های بزرگ $\{3^{-10}\}$ تا $\{3^{10}\}$ کارآمدتر باشد.
- یک حافظه کش برای ذخیره نتایج برازندگی جفت‌های (C, γ) پیاده‌سازی شده تا از محاسبات تکراری جلوگیری شود.
- موازی‌سازی با استفاده از کتابخانه `joblib` برای محاسبه برازندگی‌ها اعمال شده است.
- مکانیزم توقف زودهنگام (early stopping) برای الگوریتم‌های ممیتیک پیاده‌سازی شده تا در صورت عدم بهبود برازندگی پس از 5 نسل، اجرا متوقف شود.

توضیح منطق کد (خط به خط)

1. **وارد کردن کتابخانه‌ها:** کتابخانه‌های موردنیاز مانند `numpy` برای عملیات عددی، `pandas` برای بارگذاری داده‌ها، `sklearn` برای مدل SVM و اعتبارسنجی متقاطع، `joblib` برای موازی‌سازی و `scipy.stats` برای تست آماری وارد می‌شوند.
2. **تنظیمات اولیه:** دانه‌های تصادفی برای `random` و `numpy` تنظیم می‌شوند تا نتایج قابل تکرار باشند.

3. **بارگذاری و پیش‌پردازش داده‌ها:** مجموعه داده بیماری قلبی کلیولند از فایل CSV خوانده می‌شود. ویژگی‌ها (X) و برچسب‌ها (y) جدا شده و ویژگی‌ها با استفاده از `StandardScaler` استانداردسازی می‌شوند تا مقیاس یکسانی داشته باشند.

4. **تعریف بازه پارامترها:** بازه‌های لگاریتمی برای (C) و (γ) (از 3^{-10} تا 3^{-3}) تعریف می‌شوند، که معادل (10^{-3}) تا (10^{-1}) است.

5. **حافظه کش:** یک دیکشنری برای ذخیره نتایج برازندگی جفت‌های (C, γ) ایجاد می‌شود تا از محاسبات تکراری جلوگیری شود. مقادیر لگاریتمی تا 4 رقم اعشار گرد می‌شوند تا اندازه کش کاهش یابد.

6. **تابع برازندگی (fitness):**

- جفت (C, γ) به مقیاس اصلی (10^x) تبدیل می‌شود.
- اگر جفت در حافظه کش موجود باشد، مقدار ذخیره شده برگردانده می‌شود.
- در غیر این صورت، یک مدل SVM با پارامترهای (C) و (γ) ساخته شده و دقت آن با اعتبارسنجی متقاطع 5-تایی محاسبه می‌شود.
- نتیجه در حافظه کش ذخیره شده و برگردانده می‌شود.

7. **تابع جستجوی محلی:**

- از یک نقطه اولیه شروع می‌کند.
- برای تعداد مشخصی تکرار (100)، یک نقطه جدید با افزودن نویز گاوسی (با اندازه گام تطبیقی) ایجاد می‌شود.
- مقادیر جدید در بازه مجاز کلیپ می‌شوند.
- اگر برازندگی نقطه جدید بهتر باشد، نقطه و برازندگی به روزرسانی شده و اندازه گام 10% کاهش می‌یابد؛ در غیر این صورت، اندازه گام 10% افزایش می‌یابد.
- اندازه گام در بازه [0.05, 0.5] محدود می‌شود.

8. **الگوریتم ممتیک لامارکین:**

- یک جمعیت اولیه تصادفی از 20 فرد ایجاد می‌شود.
- برای هر نسل (حداکثر 15 نسل):
 - برازندگی همه افراد به صورت موازی محاسبه می‌شود.
 - بهترین فرد و برازندگی آن ذخیره شده و در صورت عدم بهبود پس از 5 نسل، اجرا متوقف می‌شود.
 - والدین با انتخاب تورنمنتی (3 فرد تصادفی، انتخاب بهترین) انتخاب می‌شوند.
 - عملیات تقاطع با احتمال 0.8 انجام می‌شود (ترکیب خطی دو والد).
 - جهش با احتمال 0.15 اعمال می‌شود (افزودن نویز گاوسی).
 - جستجوی محلی (5 تکرار) روی هر فرد انجام شده و تغییرات به ژنوتیپ اعمال می‌شود.
 - بهترین فرد نسل قبل به جمعیت جدید اضافه می‌شود (elitism).

9. **الگوریتم ممتیک بالدوینین:**

- مشابه لامارکین است، اما تغییرات جستجوی محلی فقط برای محاسبه برازندگی استفاده می‌شود و ژنوتیپ بدون تغییر باقی می‌ماند.

10. الگوریتم جستجوی محلی مستقل:

- مشابه تابع جستجوی محلی، اما به صورت مستقل با 100 تکرار اجرا می شود.

11. اجرای الگوریتم ها:

- هر الگوریتم 10 بار اجرا می شود.
- برای هر اجرا، زمان اجرا و نتایج (پارامترها و پرازندگی) ذخیره می شوند.

12. گزارش نتایج:

- برای هر الگوریتم، میانگین و انحراف معیار دقت، بهترین دقت، میانگین (C) و γ (در مقیاس اصلی) و میانگین زمان اجرا گزارش می شود.

13. مقایسه آماری:

- تست t دو نمونه ای بین دقت های الگوریتم ها انجام می شود تا تفاوت آماری بررسی شود.

تحلیل خروجی

خروجی اجرای کد در 10 اجرا نشان دهنده عملکرد الگوریتم ها به شرح زیر است:

1. عملکرد الگوریتم ممیتیک لامارکین:

- میانگین دقت: 0.8514 ± 0.0000 (بدون انحراف معیار، نشان دهنده پایداری بالا).
- بهترین دقت: 0.8514 (در تمام اجراها به این مقدار رسیده است).
- میانگین پارامترها: $(C = 1.6562)$, $(\gamma = 0.0083)$.
- میانگین زمان اجرا: 35.80 ثانیه.
- تحلیل: لامارکین در تمام اجراها به دقت بهینه 0.8514 رسیده و بسیار پایدار است. این نشان می دهد ترکیب الگوریتم ژنتیک با جستجوی محلی و اعمال مستقیم تغییرات به ژنوتیپ، به یافتن بهینه های سراسری کمک کرده است. با این حال، زمان اجرای بالا (35.80 ثانیه) نشان دهنده هزینه محاسباتی بیشتر به دلیل جستجوی محلی در هر نسل است.

2. عملکرد الگوریتم ممیتیک بالدوینین:

- میانگین دقت: 0.8510 ± 0.0010 (انحراف معیار بسیار کم).
- بهترین دقت: 0.8514 (در اکثر اجراها به این مقدار رسیده است).
- میانگین پارامترها: $(C = 3.0857)$, $(\gamma = 0.0167)$.
- میانگین زمان اجرا: 27.11 ثانیه.
- تحلیل: بالدوینین نیز عملکرد بسیار خوبی دارد و در اکثر اجراها به دقت 0.8514 رسیده است. انحراف معیار کم نشان دهنده پایداری است، اما کمی کمتر از لامارکین (به دلیل اجرای 6 که به 0.8414 رسیده). زمان اجرای کمتر (27.11 ثانیه) نشان می دهد که عدم اعمال تغییرات جستجوی محلی به ژنوتیپ، محاسبات را سریع تر کرده است.

3. عملکرد جستجوی محلی:

- میانگین دقت: 0.7497 ± 0.1376 (انحراف معیار بالا).
- بهترین دقت: 0.8481 (فقط در برخی اجراها به دقت های بالا رسیده).
- میانگین پارامترها: $(C = 33.4642)$, $(\gamma = 1.7837)$.

- میانگین زمان اجرا: 3.27 ثانیه.
- تحلیل: جستجوی محلی عملکرد ناپایداری دارد. در اجراهای 2، 5، 7، 8 و 9، به دقت بسیار پایین 0.5413 گیر کرده که نشان‌دهنده گیر افتادن در بهینه‌های محلی است. در اجراهای دیگر (مثل 1، 4، 6، 10)، به دقت‌های بهتری (تا 0.8481) رسیده، اما همچنان از الگوریتم‌های ممیتیک ضعیف‌تر است. زمان اجرای بسیار کم (3.27 ثانیه) مزیت اصلی آن است، اما به دلیل حساسیت به نقطه شروع اولیه، اغلب به نتایج بهینه نمی‌رسد.

مقایسه آماری:

4.

- لامارکین در مقابل بالدوینین: ($t = 1.0000$), ($p = 0.3306$). مقدار ($p > 0.05$) نشان می‌دهد تفاوت آماری معنی‌داری بین این دو الگوریتم وجود ندارد.
- لامارکین در مقابل جستجوی محلی: ($t = 2.2163$), ($p = 0.0398$). مقدار ($p < 0.05$) نشان‌دهنده برتری آماری لامارکین است.
- بالدوینین در مقابل جستجوی محلی: ($t = 2.2091$), ($p = 0.0404$). مقدار ($p < 0.05$) نشان‌دهنده برتری آماری بالدوینین است.
- تحلیل: الگوریتم‌های ممیتیک به‌طور آماری معنی‌دار بهتر از جستجوی محلی عمل کرده‌اند، اما بین لامارکین و بالدوینین تفاوت معنی‌داری وجود ندارد.

مشاهدات اضافی:

5.

- پایداری: الگوریتم‌های ممیتیک (به‌ویژه لامارکین) بسیار پایدار هستند و در تمام اجراها به دقت‌های بالا رسیده‌اند. جستجوی محلی به شدت به نقطه شروع اولیه حساس است.
- زمان اجرا: جستجوی محلی بسیار سریع‌تر است (10 برابر سریع‌تر از ممیتیک‌ها)، اما به دلیل ناپایداری، برای کاربردهایی که دقت بالا اولویت دارد، مناسب نیست.
- پارامترها: مقادیر (C) و (γ) بهینه در الگوریتم‌های ممیتیک در محدوده‌های معقولی ($C \approx 1.6-3.1$), ($\gamma \approx 0.008$) قرار دارند، اما جستجوی محلی مقادیر بسیار بزرگ‌تر ($C \approx 33.46$), ($\gamma \approx 1.78$) تولید کرده که ممکن است نشان‌دهنده بیش‌برازش یا انتخاب نادرست باشد.
- هشدار در تست آماری: هشدار Precision loss در تست t به دلیل دقت‌های یکسان لامارکین (0.8514 در تمام اجراها) ظاهر شده، اما تأثیری بر نتیجه‌گیری کلی ندارد.

نتیجه‌گیری

الگوریتم‌های ممیتیک (لامارکین و بالدوینین) به دلیل ترکیب جستجوی محلی و الگوریتم ژنتیک، عملکرد بسیار بهتری نسبت به جستجوی محلی دارند و به‌طور مداوم به دقت‌های بالا (0.8514) می‌رسند. لامارکین پایداری بیشتری دارد، اما بالدوینین به دلیل زمان اجرای کمتر (27.11 ثانیه در مقابل 35.80 ثانیه) می‌تواند گزینه بهتری برای کاربردهایی با محدودیت زمانی باشد. جستجوی محلی به دلیل سرعت بالا مناسب برای سناریوهایی با محدودیت محاسباتی است، اما به دلیل ناپایداری و گیر افتادن در بهینه‌های محلی، برای تنظیم دقیق پارامترهای SVM توصیه نمی‌شود. برای بهبود بیشتر، می‌توان تعداد تکرارهای جستجوی محلی را افزایش داد یا از استراتژی‌های پیشرفته‌تر (مثل Simulated Annealing) برای فرار از بهینه‌های محلی استفاده کرد.